

Eventos no Navegador

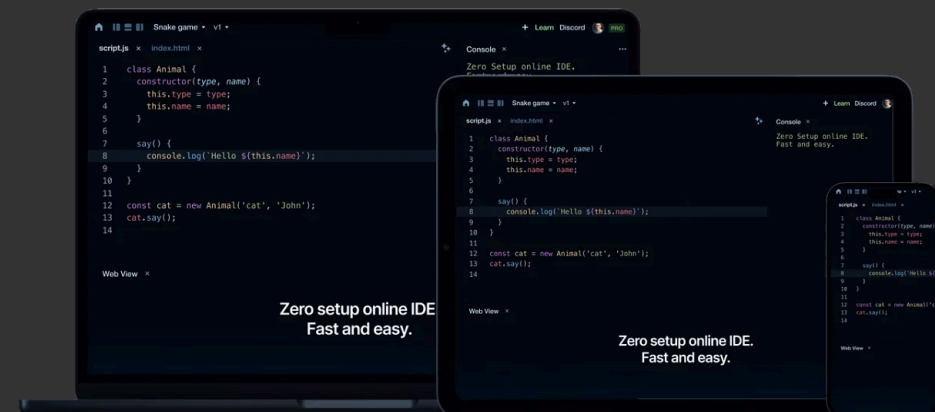
Alguns programas trabalham com entrada direta do usuário, como ações de mouse e teclado, por exemplo. O navegador permite essas interações e fornece uma série de métodos de "capturarmos" qual foi a interação do usuário com a página para reagirmos à ela., e é o que exploraremos agora.



Event listeners (Manipuladores de evento)

Os manipuladores (ou ouvintes) de eventos, mais comumente conhecidos como *event listeners* em inglês, precisam estar vinculados à algum elemento do navegador. Portanto, invocamos o método `addEventListener()` a partir de um objeto, mesmo que seja o `window`, por exemplo:

```
window.addEventListener('click', function () { console.log('Você clicou?'); })
```



Os dois parâmetros essenciais do método são o evento (string) e o *callback*, a função que será invocada quando o evento ocorrer. No caso passamos uma função anônima definida na própria invocação do método, mas poderíamos atribuir uma função nomeada:

```
windows.addEventListener('click', clicouNaPagina); function clicouNaPagina()
{ console.log('Você clicou?'); }
```

Evento em elemento

Se quisermos adicionar um evento a um elemento da página em específico, podemos consultar esse elemento com as funções de consulta (querySelector, getElementById, etc), e a partir do objeto que referencia o nó do nosso elemento no DOM, nós invocamos o método `addEventListener`:

```
const botao = document.querySelector('#botao-principal'); botao.addEventListener('click', function () { console.log('Você clicou no botão principal!'); })
```

Atribuindo eventos propriedade do objeto

Os objetos que referenciam nós do DOM possuem propriedades relativas à eventos do navegador, que mudam levemente de nome em relação aos identificadores que usamos no `addEventListener`. Por exemplo, poderíamos adicionar um evento de clique como fizemos no botão anterior, usando a propriedade `onclick`, associando nossa função à ela:

```
const botao = document.querySelector('#botao-principal'); botao.onclick = function () { console.log('Você clicou no botão principal!'); }
```

Limitações de adicionar eventos à propriedades

Muitas vezes queremos adicionar vários *event listeners* à uma ação em um determinado objeto (por exemplo, ao adicionar um produto no carrinho tanto calcular o novo preço total, como fazer uma mudança visual na interface da página), e com propriedades de objeto só temos a possibilidade de associar um único *event listener*.

```
const h1 = document.querySelector('h1.title'); h1.addEventListener('click', mensagemEventoA); h1.addEventListener('click',  
mensagemEventoB); function mensagemEventoA() { console.log('Essa é a mensagem do evento A!'); } function mensagemEventoB() {  
console.log('Essa é a mensagem do evento B!'); }
```

Remover eventos de objetos

O método `removeEventListener`, chamado com argumentos similares a `addEventListener`, remove um manipulador.

```
let button = document.querySelector("button"); function umaVez() { console.log("Clicado, já era."); button.removeEventListener("click", umaVez); } button.addEventListener("click", umaVez);
```

Objetos de evento

Embora tenhamos ignorado isso até agora, funções manipuladoras de evento recebem um argumento: o objeto de evento. Esse objeto contém informações adicionais sobre o evento. Por exemplo, se quisermos saber qual botão do mouse foi pressionado, podemos olhar a propriedade `button` do objeto de evento.

```
let button = document.querySelector("button"); button.addEventListener("mousedown", function (event) { if (event.button == 0) { console.log("Botão esquerdo"); } else if (event.button == 1) { console.log("Botão central"); } else if (event.button == 2) { console.log("Botão direito"); } });
```

A informação armazenada em um objeto de evento difere por tipo de evento. A propriedade `type` do objeto sempre contém uma string identificando o evento (como `"click"` ou `"mousedown"`).

Ações padrão

Muitos eventos têm uma ação padrão. Se você clicar em um link, será levado ao destino do link. Se pressionar a seta para baixo, o navegador rolará a página para baixo. Se clicar com o botão direito, verá um menu de contexto. E assim por diante.

Para a maioria dos tipos de eventos, os manipuladores de evento JavaScript são chamados antes que o comportamento padrão ocorra. Se o manipulador não quiser que esse comportamento normal aconteça, tipicamente porque já cuidou de manipular o evento, ele pode chamar o método `preventDefault` no objeto de evento.

```
<a href="https://developer.mozilla.org/">MDN</a>
```

 **Alerta:** Tente não fazer coisas assim sem uma razão realmente boa. Será desagradável para as pessoas que usam sua página quando o comportamento esperado é quebrado.

Eventos de tecla

Quando uma tecla no teclado é pressionada, seu navegador dispara um evento "keydown". Quando ela é solta, você recebe um evento "keyup".

```
window.addEventListener("keydown", event => { if (event.key == "v") { document.body.style.background = "violet"; } });  
window.addEventListener("keyup", event => { if (event.key == "v") { document.body.style.background = ""; } });
```



Apesar do nome, "keydown" dispara não apenas quando a tecla é fisicamente pressionada. Quando uma tecla é pressionada e mantida, o evento dispara novamente toda vez que a tecla repete. Às vezes você precisa ter cuidado com isso. Por exemplo, se você adicionar um botão ao DOM quando uma tecla é pressionada e removê-lo quando a tecla é solta, pode acidentalmente adicionar centenas de botões quando a tecla é mantida pressionada por mais tempo. Exemplo:

```
window.addEventListener("keydown", event => { let p =  
document.createElement('p'); p.textContent = 'Novo parágrafo';  
document.querySelector('body').appendChild(p); });
```

O exemplo anterior observa a propriedade `key` do objeto de evento para ver sobre qual tecla o evento é. Essa propriedade contém uma string que, para a maioria das teclas, corresponde ao que pressionar aquela tecla digitaria. Para teclas especiais como `enter`, ela contém uma string que nomeia a tecla ("`Enter`", neste caso). Se você segura `shift` enquanto pressiona uma tecla, isso também pode influenciar o nome da tecla — "`v`" se torna "`V`", e "`1`" pode se tornar "`!`", se é isso que pressionar `shift-1` produz no seu teclado.

Teclas modificadoras

Teclas modificadoras como shift, ctrl, alt e meta (command no Mac) geram eventos de tecla assim como teclas normais. Ao procurar combinações de teclas, você também pode descobrir se essas teclas estão pressionadas olhando as propriedades `shiftKey`, `ctrlKey`, `altKey` e `metaKey` dos eventos de teclado e mouse.

Press Control-Space to **continue**.

Foco em eventos de tecla

O nó DOM onde um evento de tecla se origina depende do elemento que tem foco quando a tecla é pressionada. A maioria dos nós não pode ter foco a menos que você lhes dê um atributo `tabindex`, mas coisas como links, botões e campos de formulário podem.

Conteúdo digitado

Para notar quando algo foi digitado, elementos nos quais você pode digitar, como as tags e , disparam eventos "input" sempre que o usuário muda seu conteúdo, isso é muito melhor do que ficar verificando eventos de tecla nestes casos.

Eventos de ponteiro

Existem atualmente duas formas amplamente usadas de apontar para coisas em uma tela: mouses e telas de toque. Estes produzem diferentes tipos de eventos.

Cliques do mouse

Pressionar um botão do mouse faz uma série de eventos dispararem.

Os eventos "mousedown" e "mouseup" são similares a "keydown" e "keyup" e disparam quando o botão é pressionado e solto. **Eles acontecem nos nós DOM que estão imediatamente abaixo do ponteiro do mouse quando o evento ocorre.**

mouseup e click

Após o evento "mouseup", um evento "click" dispara no nó mais específico que continha tanto o pressionamento quanto a liberação do botão. Por exemplo, se eu pressionar o botão do mouse em um parágrafo e depois mover o ponteiro para outro parágrafo e soltar o botão, o evento "click" acontecerá no elemento que contém ambos os parágrafos.

dblclick

Se dois cliques acontecem próximos um do outro, um evento "dblclick" (duplo clique) também dispara, após o segundo evento de clique.

Coordenadas x e y

Para obter informações precisas sobre o local onde um evento de mouse aconteceu, você pode olhar suas propriedades clientX e clientY, que contêm as coordenadas do evento (em pixels) relativas ao canto superior esquerdo da janela, ou pageX e pageY, que são relativas ao canto superior esquerdo do documento inteiro (que pode ser diferente quando a janela foi rolada).

O programa a seguir implementa um aplicativo de desenho primitivo. Toda vez que você clicar no documento, ele adiciona um ponto sob o ponteiro do mouse.

```
body { height: 200px; background: beige; } .dot { height: 8px; width: 8px; border-radius: 4px; /* arredonda cantos */ background: teal; position: absolute; }
```

```
window.addEventListener("click", event => { let dot = document.createElement("div"); dot.className = "dot"; dot.style.left = (event.pageX - 4) + "px"; dot.style.top = (event.pageY - 4) + "px"; document.body.appendChild(dot); });
```

Movimento do mouse

Toda vez que o ponteiro do mouse se move, um evento "mousemove" dispara. Esse evento pode ser usado para rastrear a posição do mouse. Uma situação comum em que isso é útil é ao implementar alguma forma de funcionalidade de arrastar com o mouse.

Como exemplo, o programa a seguir exibe uma barra e configura manipuladores de evento para que arrastar para a esquerda ou direita nessa barra a torne mais estreita ou mais larga:

Arraste a barra para aumentar seu comprimento:

```
let ultimaPosicaoX; // Rastreia a última posição X observada do mouse let barra = document.querySelector("div"); barra.addEventListener("mousedown", function(event) { // Caso o botão esquerdo estiver pressionado if (event.button == 0) { ultimaPosicaoX = event.clientX; // Pega o valor da última posição de X window.addEventListener("mousemove", moveu); // Adiciona um ouvinte de evento para mousemove com callback para a função moveu event.preventDefault(); // Prevenir seleção } }); function moveu(event) { // Caso nenhum botão estiver pressionado, remover evento if (event.buttons == 0) { window.removeEventListener("mousemove", moveu); } else { let distancia = event.clientX - ultimaPosicaoX; let novoComprimento = Math.max(10, barra.offsetWidth + dist); barra.style.width = novoComprimento + "px"; ultimaPosicaoX = event.clientX; } }
```

Note que o manipulador "mousemove" é registrado na janela inteira. Mesmo se o mouse sair da barra durante o redimensionamento, enquanto o botão estiver pressionado, ainda queremos atualizar seu tamanho.

Devemos parar de redimensionar a barra quando o botão do mouse é solto. Para isso, podemos usar a propriedade buttons (note o plural), que nos diz sobre os botões que estão atualmente pressionados. Quando é 0, nenhum botão está pressionado. Quando botões estão pressionados, o valor da propriedade buttons é a soma dos códigos desses botões — o botão esquerdo tem código 1, o direito 2 e o do meio 4. Com os botões esquerdo e direito pressionados, por exemplo, o valor de buttons será 3.

Note que a ordem desses códigos é diferente da usada por button, onde o botão do meio vinha antes do direito. Como mencionado, consistência não é um ponto forte da interface de programação do navegador.



Referências

HAVERBEKE, Marijn. **Manipulando Eventos**. In: _____. **Eloquent JavaScript: a modern introduction to programming**. 4. ed. San Francisco: No Starch Press, 2024. cap. 15.